MANJARA CHARITABLE TRUST

SMT. SUSHILADEVI DESHMUKH COLLEGE OF ARTS,

SCIENCE AND COMMERCE, Airoli, Sector-4, Navi Mumbai-400 708

Date:

CERTIFICATE

This is to Certify that **Mr.Rudra Dubey** seat no.**06** of **TYBSC CS** Semester **V** has completed the practical work in the subject of **"Information & Network Security"** during the academic year 2025-26 under the guidance of **Asst.Prof Soni Yadav** being the partial requirement for the fulfilment of the curriculum of Degree of Bachelor of Computer Science, University of Mumbai.

Signature of Internal Guide **Signature of HOD**

Signature of External **Principal**

College Seal

INS PRACTICAL 2025-26

INDEX

DATE	TOPIC	SIGN
1	Implementing Substitution and Transposition Ciphers	
2	RSA Encryption and Decryption	
3	Key Exchange using Diffie-Hellman	
4	Message Authentication Codes	
5	Malware Analysis and Detection	
6	Web Security with SSL/TLS	
7	IP Security (IPSec) Configuration	
8	Firewall Configuration and Rule-based Filtering	

INS PRACTICAL 2025-26

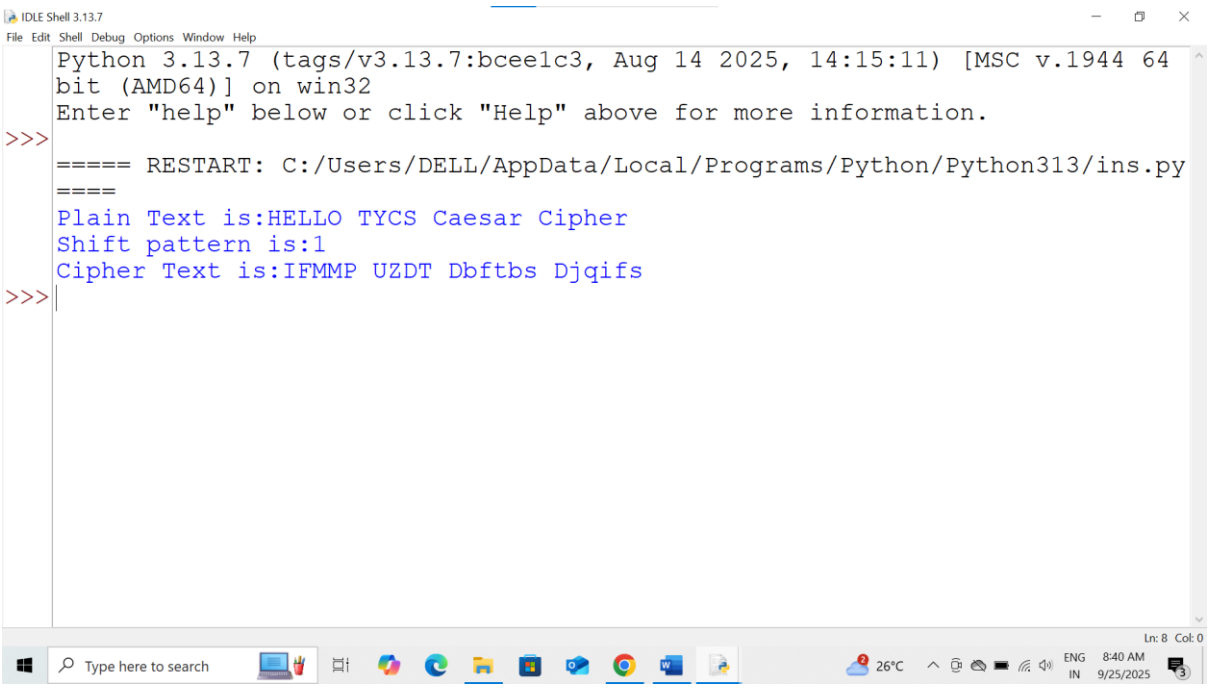
PARTICAL-1

Q. Implement Substitution and Transposition cipher.

A) Design and implement algorithm to encrypt and decrypt messages using classical substitution and transposition techniques.

```
def encrypt_text(plaintext,n):
   ans=""
   for i in range(len(plaintext)):
      ch=plaintext[i]
      if ch==" ":
         ans+=" "
      elif(ch.isupper()):
         ans+=chr((ord(ch)+n-65)%26+65)
      else:
         ans+=chr((ord(ch)+n-97)%26+97)
   return ans
plaintext="HELLO TYCS Caesar Cipher"
n=1
print("Plain Text is:"+plaintext)
print("Shift pattern is:"+str(n))
print("Cipher Text is:"+encrypt_text(plaintext,n))
```

OUTPUT :



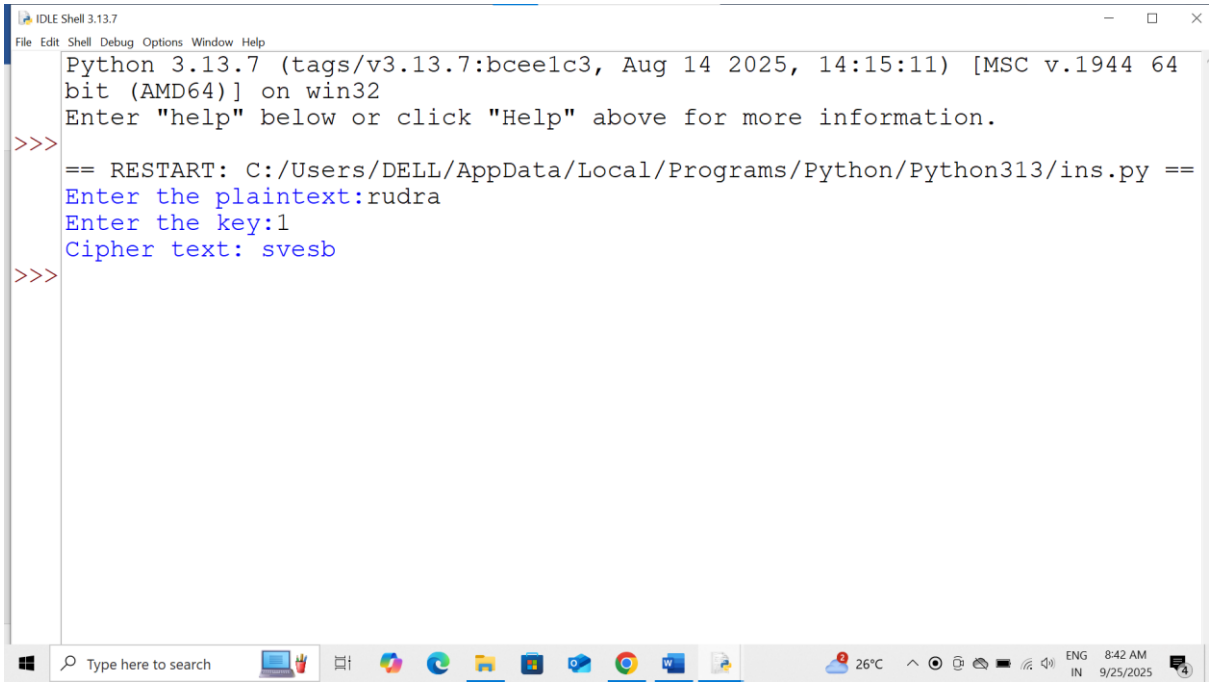
Name: Rudra Dubey Roll no: 06 Class: TYCS

INS PRACTICAL 2025-26

B) Monoalphabetic Cipher-Additive Cipher.

```
def encrypt(message,key):
    cipher=""
    for i in message:
        if i.isupper():
            cipher+=chr((ord(i)+key-65)%26+65)
        elif i.islower():
            cipher+=chr((ord(i)+key-97)%26+97)
        else:
            cipher+=" "
    return cipher
message = input("Enter the plaintext:")
key=input("Enter the key:")
if key.isupper():
    key=ord(key)-65
elif key.islower():
    key=ord(key)-97
else:
    key=int(key)
print("Cipher text:",encrypt(message,key))
```

OUTPUT:

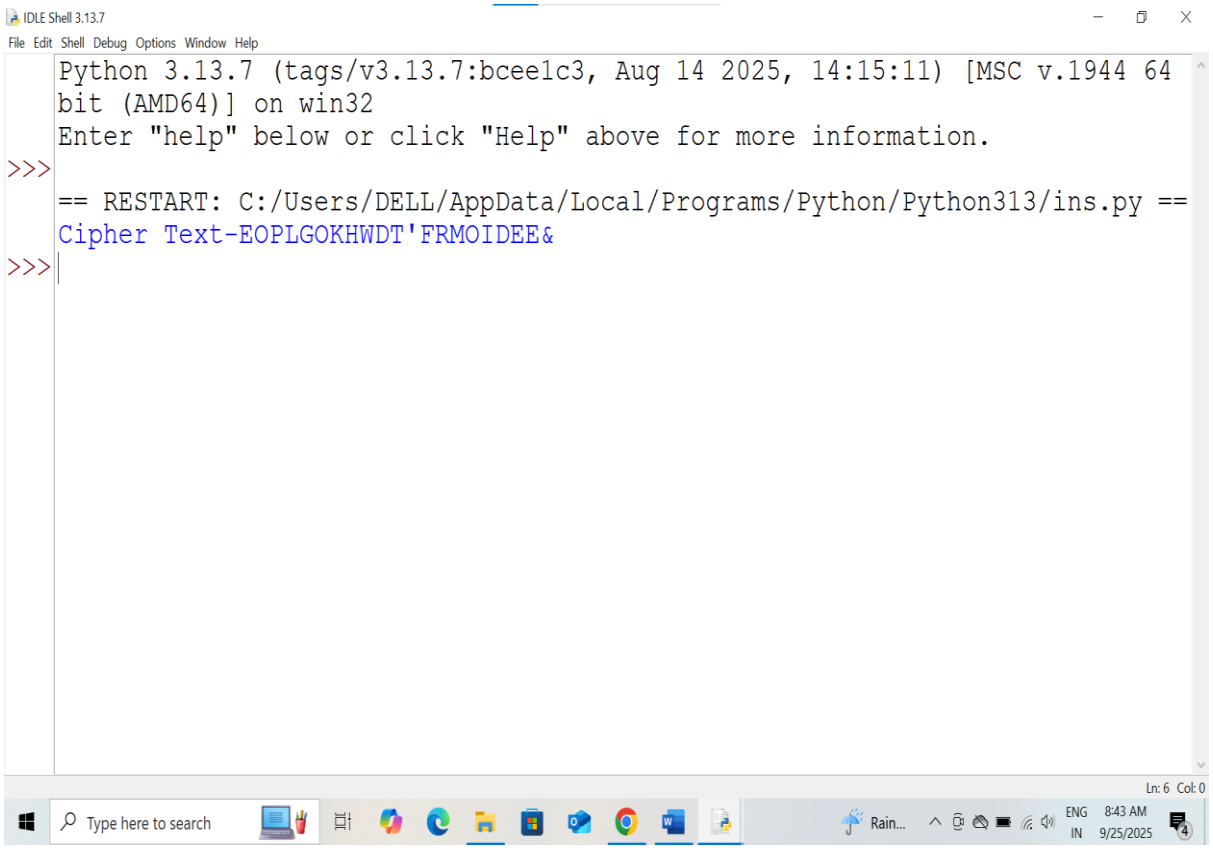


INS PRACTICAL 2025-26

C) Vernam Cipher.

```
def stringEncryption(text,key):
    cipherText=""
    cipher=[]
    for i in range(len(key)):
        cipher.append(ord(text[i])-ord('A')+ord(key[i])-ord('A'))
    for i in range(len(key)):
        if cipher[i]>25:
            cipher[i]=cipher[i]-26
    for i in range(len(key)):
        x=cipher[i]+ord('A')
        cipherText+=chr(x)
    return cipherText
plainText="Sachinkumar Rajkumar Nishad"
key="MONEYBANKDCHORDEORENG"
encryptedText=stringEncryption(plainText.upper(),key.upper())
print("Cipher Text-"+encryptedText)
```

OUTPUT:

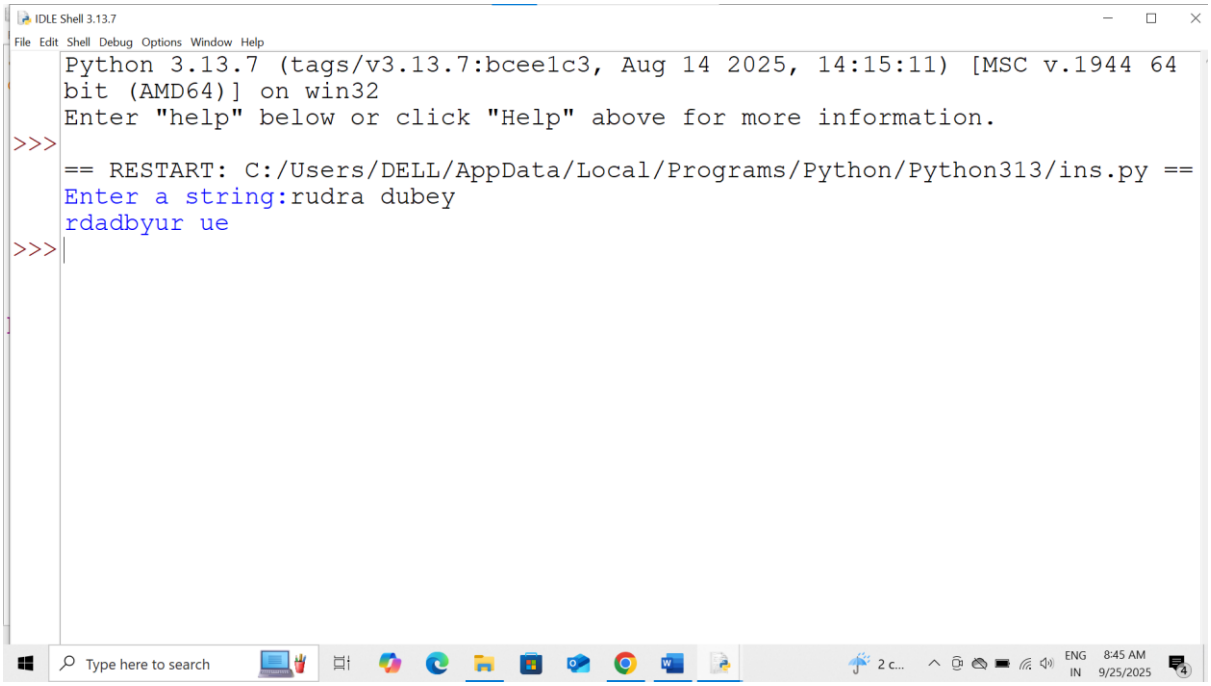


INS PRACTICAL 2025-26

D) Railfence Cipher

```
string=input("Enter a string:")
def RailFence(txt):
   result=""
   for i in range(len(string)):
      if(i%2==0):
         result+=string[i]
   for i in range(len(string)):
      if (i%2!=0):
         result+=string[i]
   return result
print(RailFence(string))
```

OUTPUT:

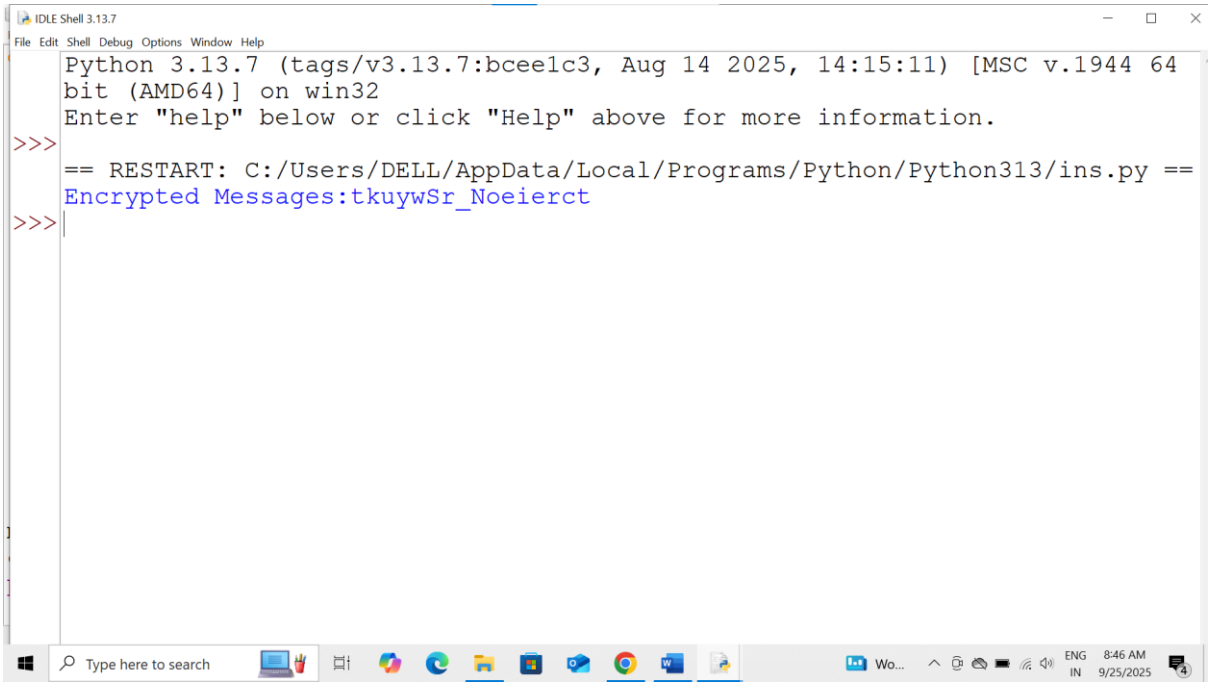


INS PRACTICAL 2025-26

E) Simple Columnar Technique

```
import math
key="TYCS"
def encryptMessage(msg):
   cipher=""
   k_indx=0
   msg_len=float(len(msg))
   msg_lst=list(msg)
   key_lst=sorted(list(key))
   col=len(key)
   row=int(math.ceil(msg_len/col))
   fill_null=int((row*col)-msg_len)
   msg_lst.extend('_'*fill_null)
   matrix=[msg_lst[i:i+col]
         for i in range(0,len(msg_lst),col)]
   for _ in range(col):
      curr_idx=key.index(key_lst[k_indx])
      cipher+=''.join([row[curr_idx]
                  for row in matrix])
      k_indx+=1
   return cipher
msg="NetworkSecurity"
cipher=encryptMessage(msg)
print("Encrypted Messages:{}".format(cipher))
```

OUTPUT:



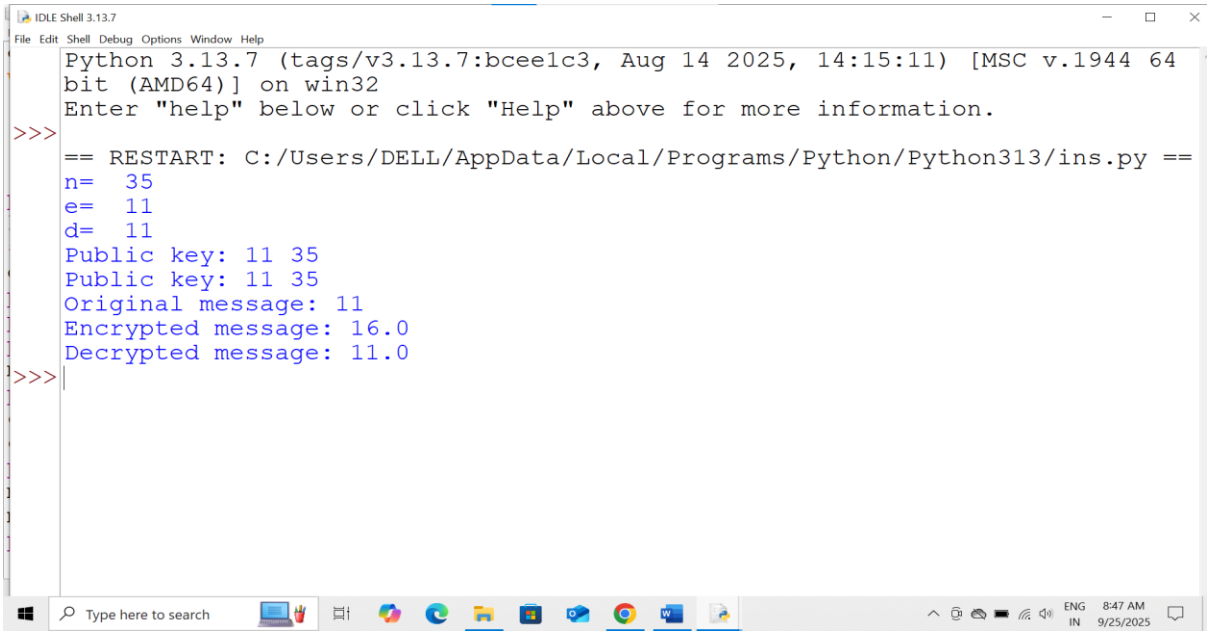
INS PRACTICAL 2025-26

PRATICAL-2

Q. RSA Encryption and Decryption.

```
import math
p=5  #3
q=7
n=p*q
print("n= ",n)
phi =(p-1)*(q-1)
e=11  #2
while(e<phi):
   if (math.gcd(e,phi)==1):
      break
   else:
      e +=1
print("e= ",e)
k=2
##d=((k*phi)+1)/e
d= 11
print("d= ",d)
print("Public key:",e,n)
print("Public key:",d,n)
msg=11
print("Original message:",msg)
c=pow(msg,e)
c= math.fmod(c,n)
print("Encrypted message:",c)
m=pow(c,d)
m=math.fmod(m,n)
print("Decrypted message:",m)
```

OUTPUT:



Name: Rudra Dubey Roll no: 06 Class: TYCS

INS PRACTICAL 2025-26

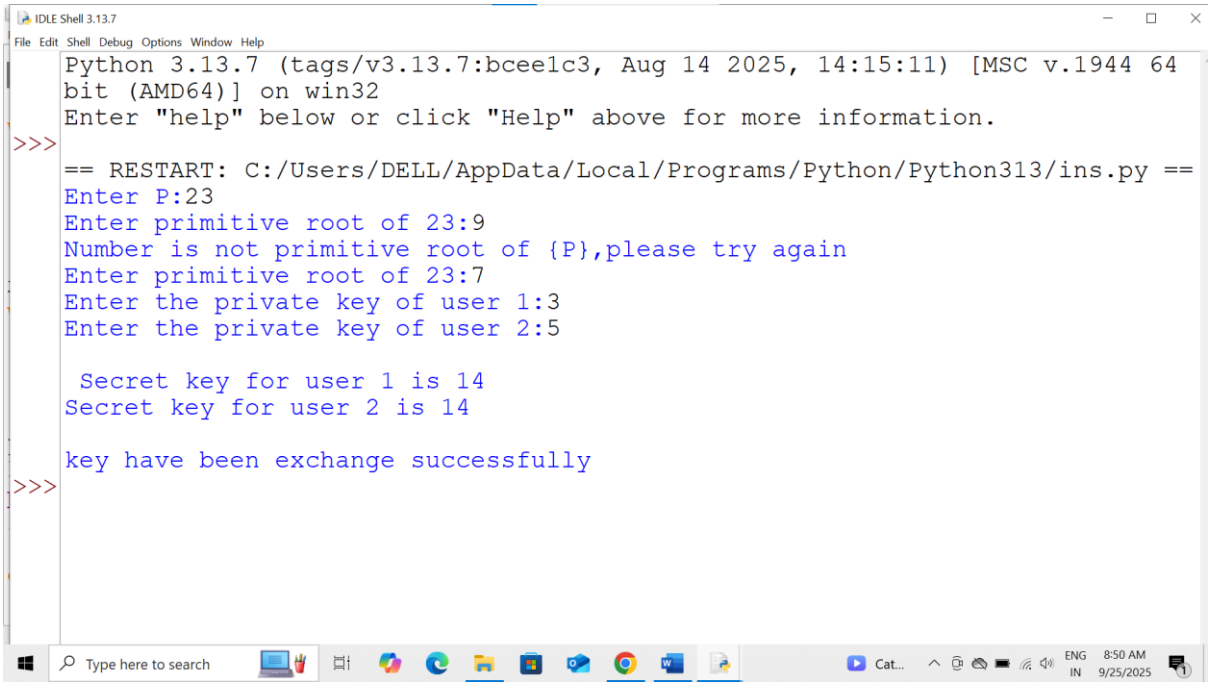
PRATICAL-3

Q. Key Exchange using Diffie-Hellman.

```
def prime_checker(p):
    if p<1:
        return -1
    elif p>1:
        if p==2:
            return 1
        for i in range(2,p):
            if p%i==0:
                return -1
            return 1
def primitive_check(g,p,L):
    for i in range(1,p):
        L.append(pow(g,i)%p)
    for i in range(1,p):
        if L.count(i)>1:
            L.clear()
            return -1
        return 1
l=[]
while 1:
    P=int(input("Enter P:"))
    if prime_checker(P)==-1:
        print("Number is not prime, please enter again!")
        continue
    break
while 1:
    G=int(input(f"Enter primitive root of {P}:"))
    if primitive_check(G,P,l)==-1:
        print("Number is not primitive root of {P},please try again")
        continue
    break
x1,x2=int(input("Enter the private key of user 1:")),int(input("Enter the private key of user 2:"))
while 1:
    if x1>=P or x2>=P:
        print(f"Private key of both the user should be less than {P}!")
        continue
    break
y1,y2=pow(G,x1)%P,pow(G,x2)%P
k1,k2=pow(y2,x1)%P,pow(y1,x2)%P
print(f"\n Secret key for user 1 is {k1}\nSecret key for user 2 is {k2}\n")
if k1==k2:
    print("key have been exchange successfully")
else:
    print("key have not been exchange successfully")
```

INS PRACTICAL 2025-26

OUTPUT:



Name: Rudra Dubey Roll no: 06 Class: TYCS

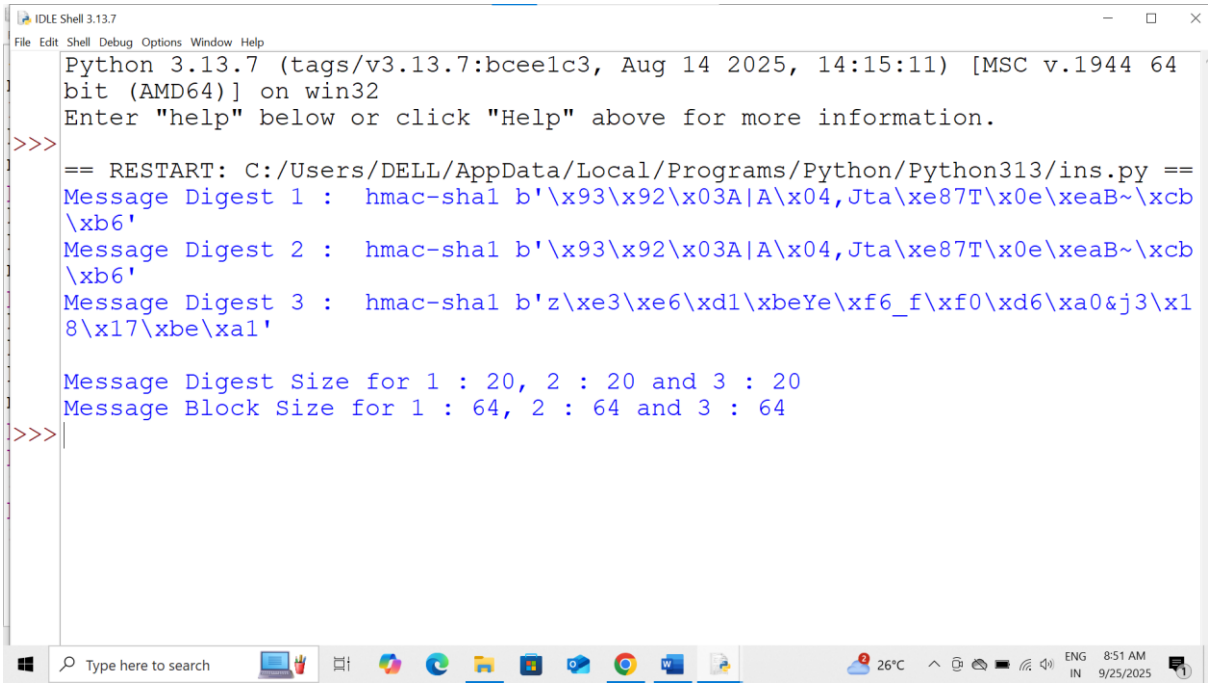
INS PRACTICAL 2025-26

PRATICAL-4

Q. Message Authentication Codes
Implement algorithms to generate and verify message authentication code(MAC) for ensuring data integrity and authenticity.

```
import hmac
message = "Welcome to TYCS."
key= "INS"
hmac1=hmac.new(key=key.encode(), msg=message.encode(), digestmod="sha1")
message_digest1 = hmac1.digest()
print("Message Digest 1 : ",hmac1.name, message_digest1)
hmac2 = hmac.new(key=key.encode(), digestmod="sha1")
hmac2.update(bytes(message, encoding="utf-8"))
message_digest2 = hmac2.digest()
print("Message Digest 2 : ",hmac2.name, message_digest2)
hmac3 = hmac.new(key=key.encode(), digestmod="sha1")
hmac3.update(bytes("Welcome to ", encoding="utf-8"))
hmac3.update(bytes("CoderzColumn.", encoding="utf-8"))
message_digest3 = hmac3.digest()
print("Message Digest 3 : ",hmac3.name, message_digest3)
print("\nMessage Digest Size for 1 : {}, 2 : {} and 3 : {}".
format(hmac1.digest_size, hmac2.digest_size,hmac3.digest_size,))
print("Message Block Size for 1 : {}, 2 : {} and 3 : {}".
format(hmac1.block_size, hmac2.block_size,hmac3.block_size,))
```

OUTPUT:

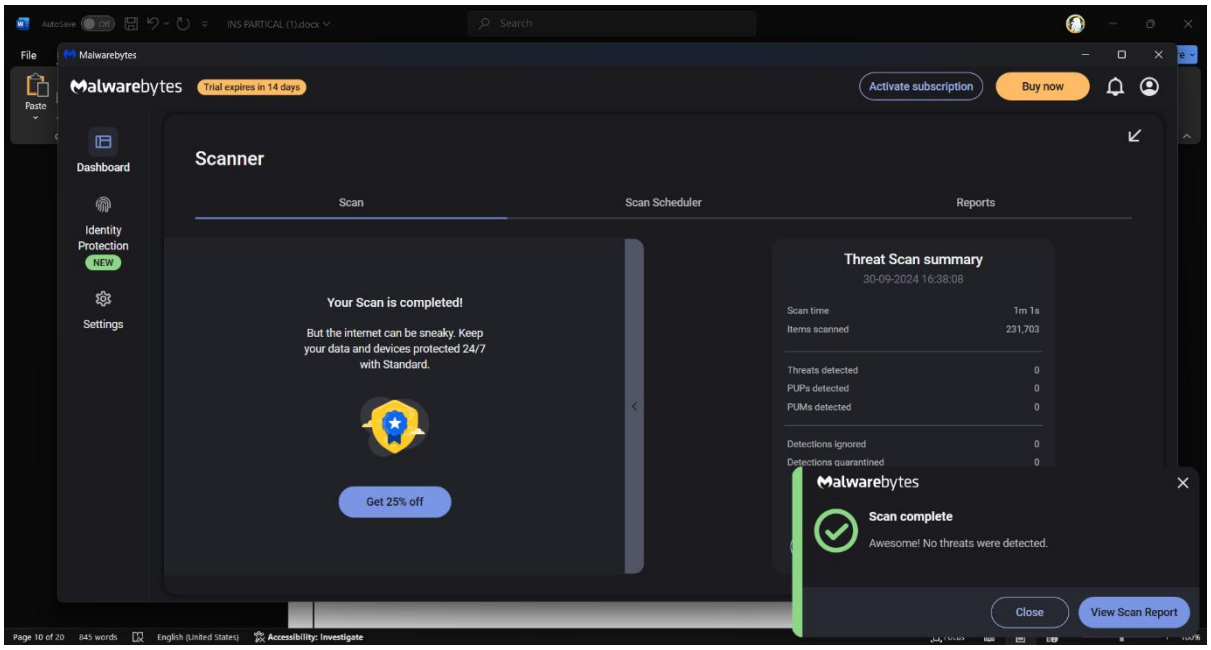


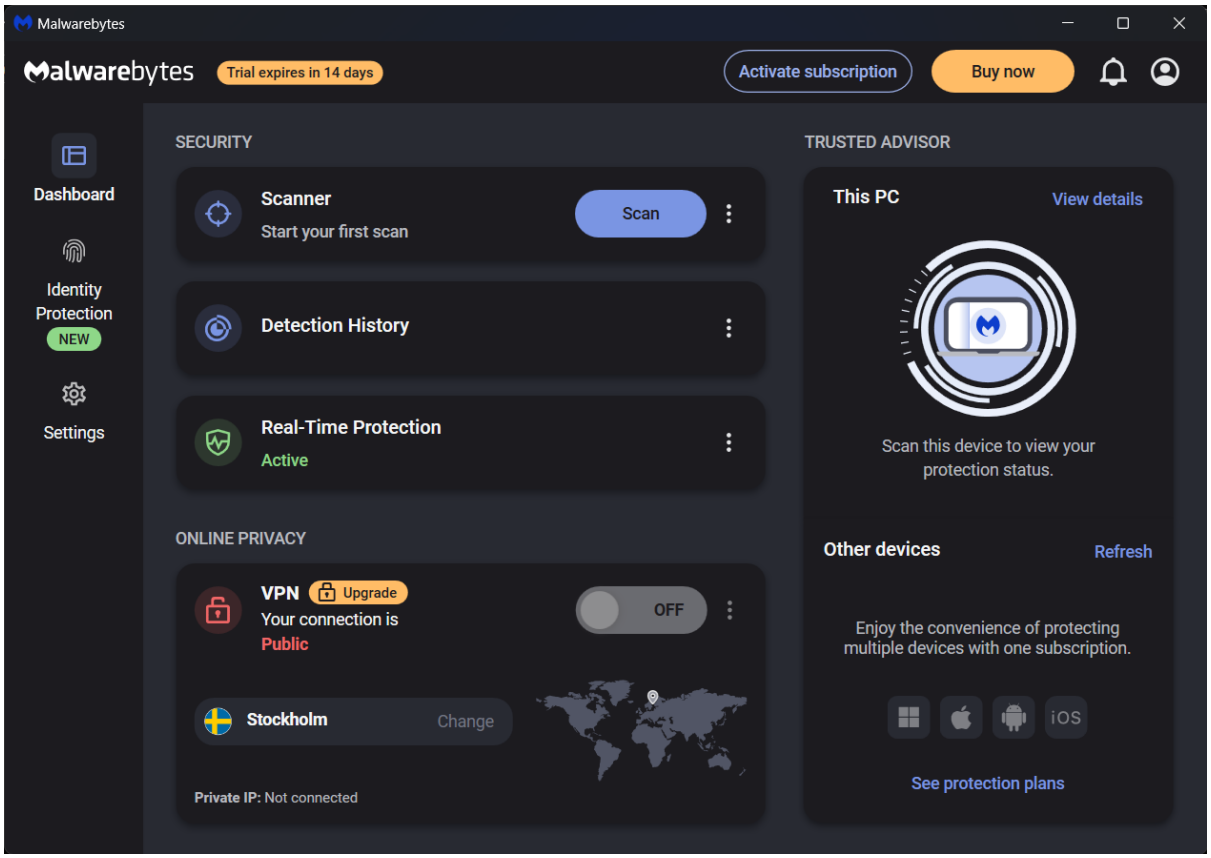
Practical 5

Q. Malware Analysis and Detection.

INS PRACTICAL 2025-26

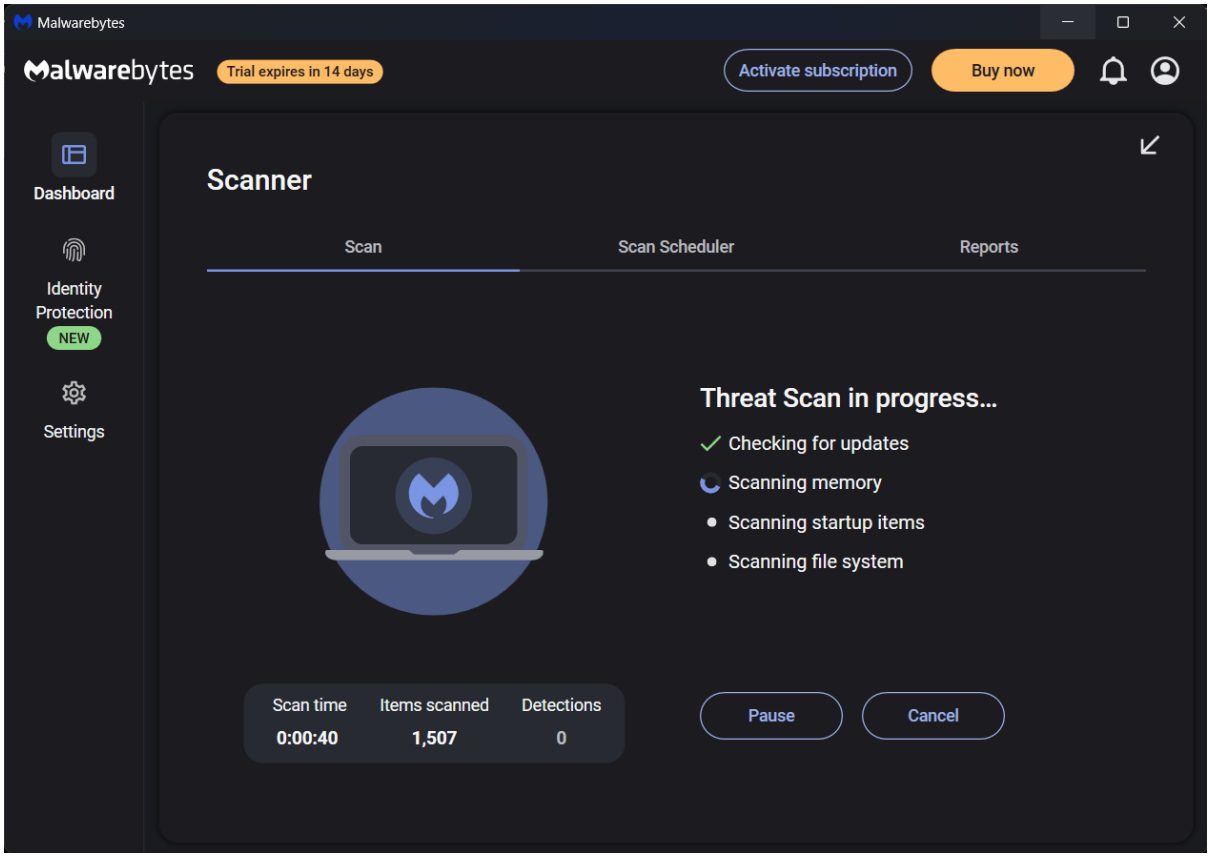
Analyze and identify malware samples using antivirus tools, analyze their behaviour , and develop countermeasures to mitigate their impact.





Name: Rudra Dubey Roll no: 06 Class: TYCS

INS PRACTICAL 2025-26



Name: Rudra Dubey Roll no: 06 Class: TYCS

INS PRACTICAL 2025-26

Practical 6

Web Security with SSL/TLS

1.Open Google Chrome.

2.Press Alt + f and click on settings.

3.Select the show advance settings option.

4.Scroll down to the network section and click on change proxy settings button.

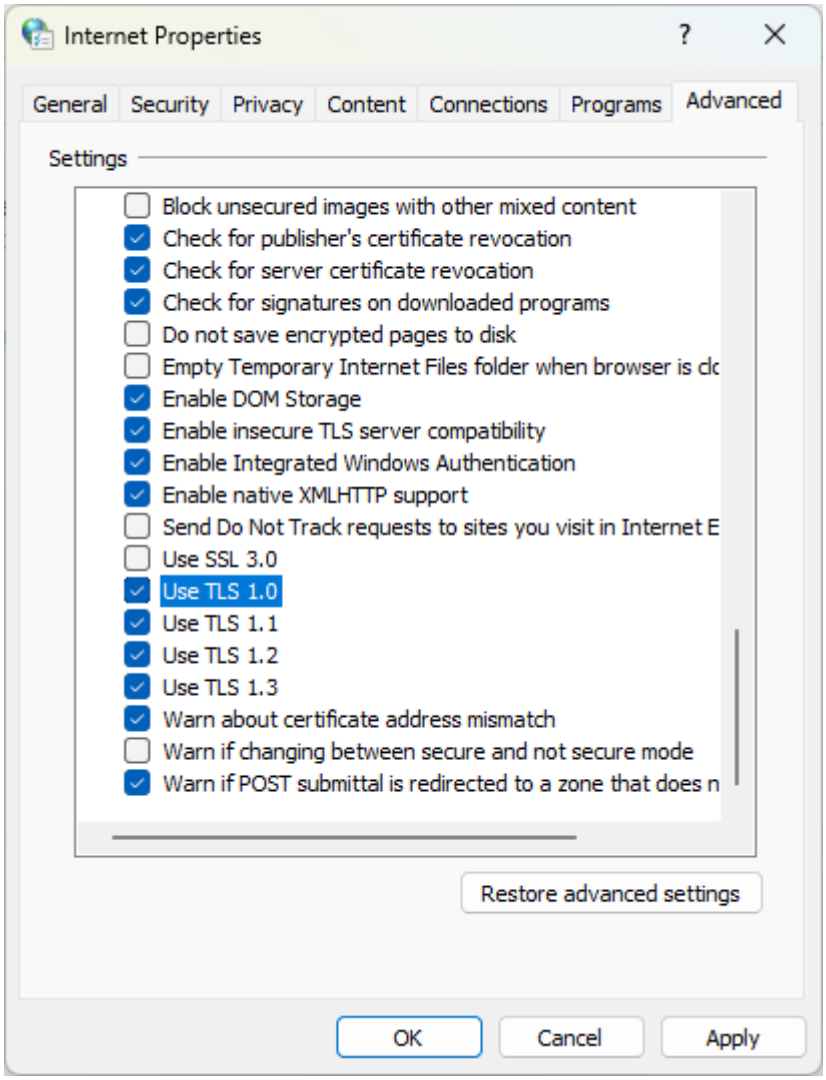
5.Now go to the Advanced tab.

6.Scroll down to the security category.

7.Now check the boxes for your TLS/SSL version.

8.Click Ok.

9.Close the browser and open it again.



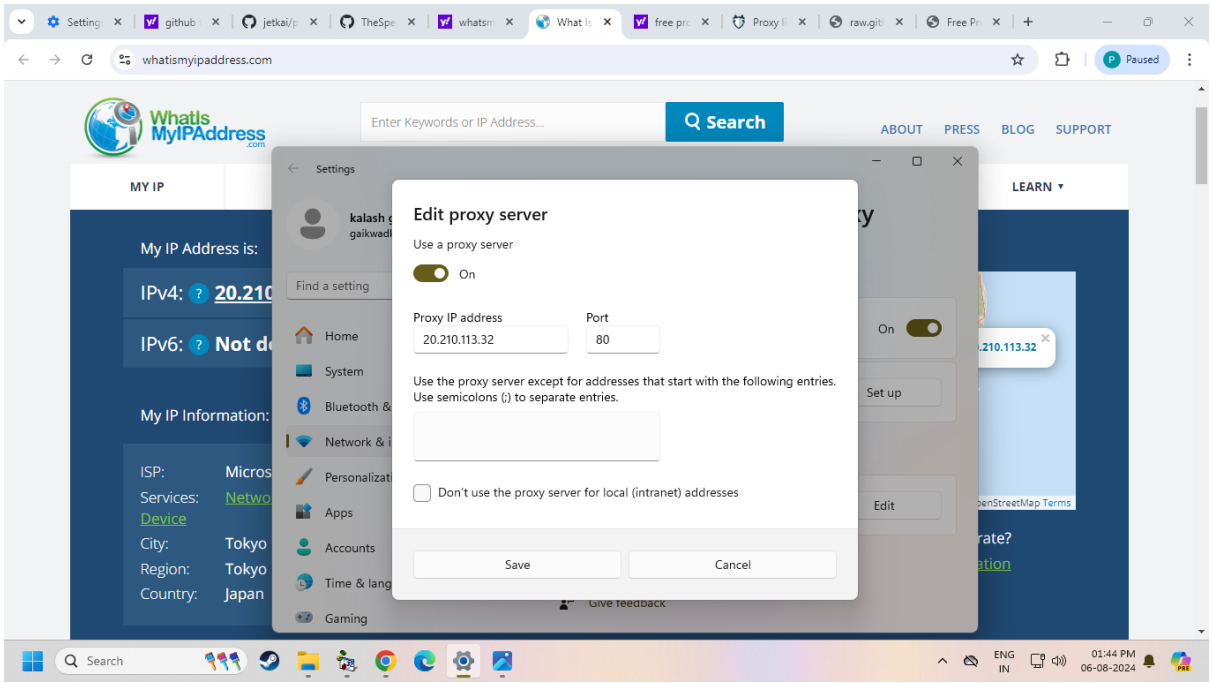
Name: Rudra Dubey Roll no: 06 Class: TYCS

INS PRACTICAL 2025-26

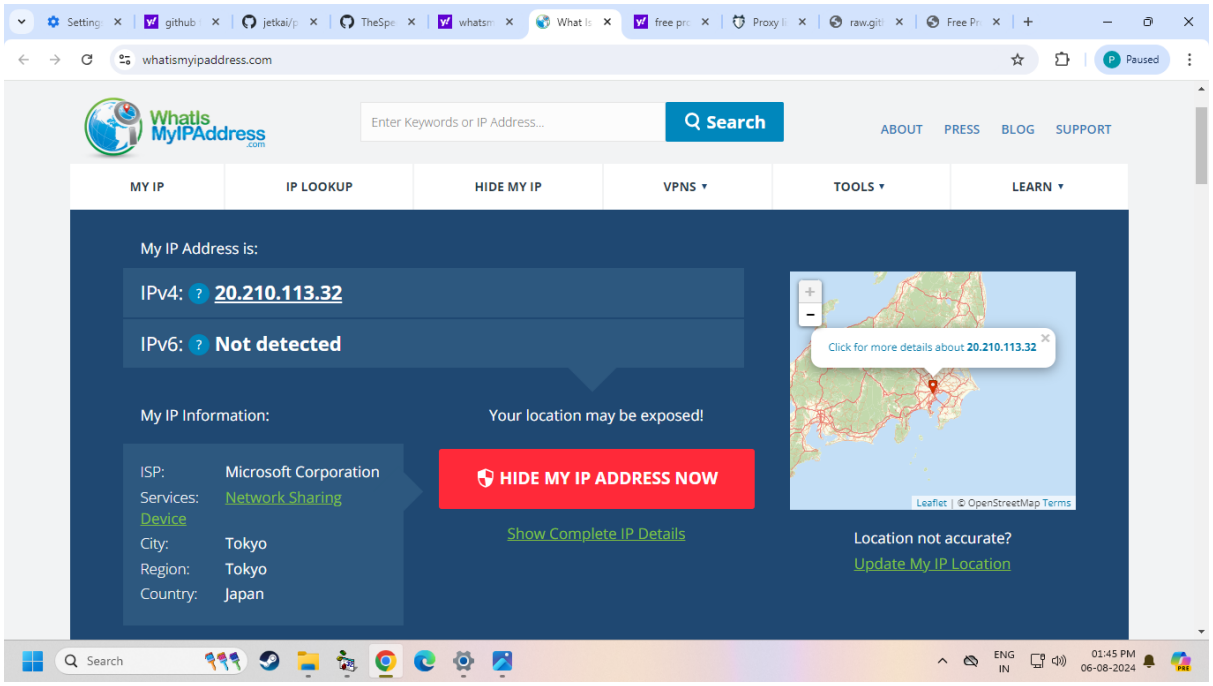
Practical 7

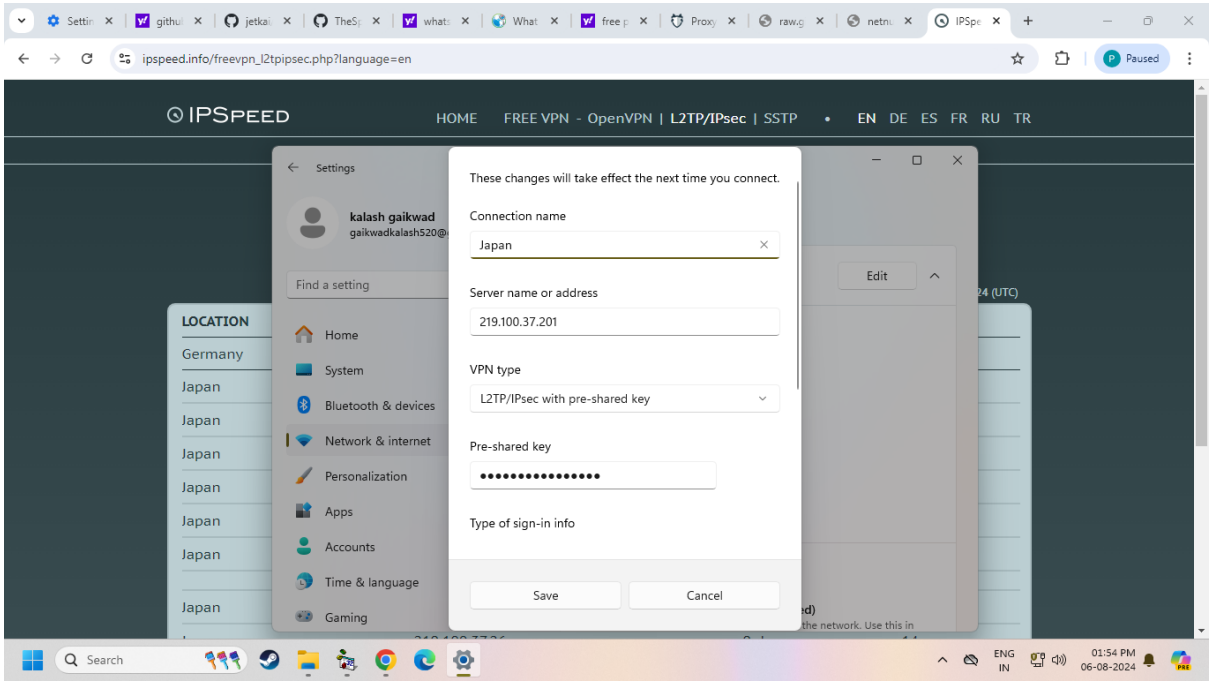
IP Security (IPSec) Configuration.
Configure IPsec on network devices to provide secure communication and protect against unauthorized access and attacks.

1. Right-click on the wireless/network icon in your system tray.
2. Select Open Network and Sharing Center. Or, if using Windows 10 version 1709 or newer, select Open Network & Internet settings, then click Network and Sharing Center.
3. Click Set up a new connection or network.
4. Select Connect to a workplace and click Next.
5. Click Use my Internet connection (VPN).
6. Enter Your VPN Server IP in the Internet address field.
7. Enter anything you like in the Destination name field, and then click Create.
8. Return to Network and Sharing Center. On the left, click Change adapter settings.
9. Right-click on the new VPN entry and choose Properties.
10. Click the Security tab. Select Layer 2 Tunneling Protocol with IPsec (L2TP/IPSec) for the type of VPN.
11. Click Allow these protocols. Check the Challenge Handshake Authentication Protocol (CHAP) and Microsoft CHAP Version 2 (MS-CHAP v2) checkboxes.
12. Click Advanced settings.
13. Select Use preshared key for authentication and enter Your VPN IPsec PSK for the key.
14. Click OK to close the Advanced settings.
15. Click OK to save the VPN connection details.



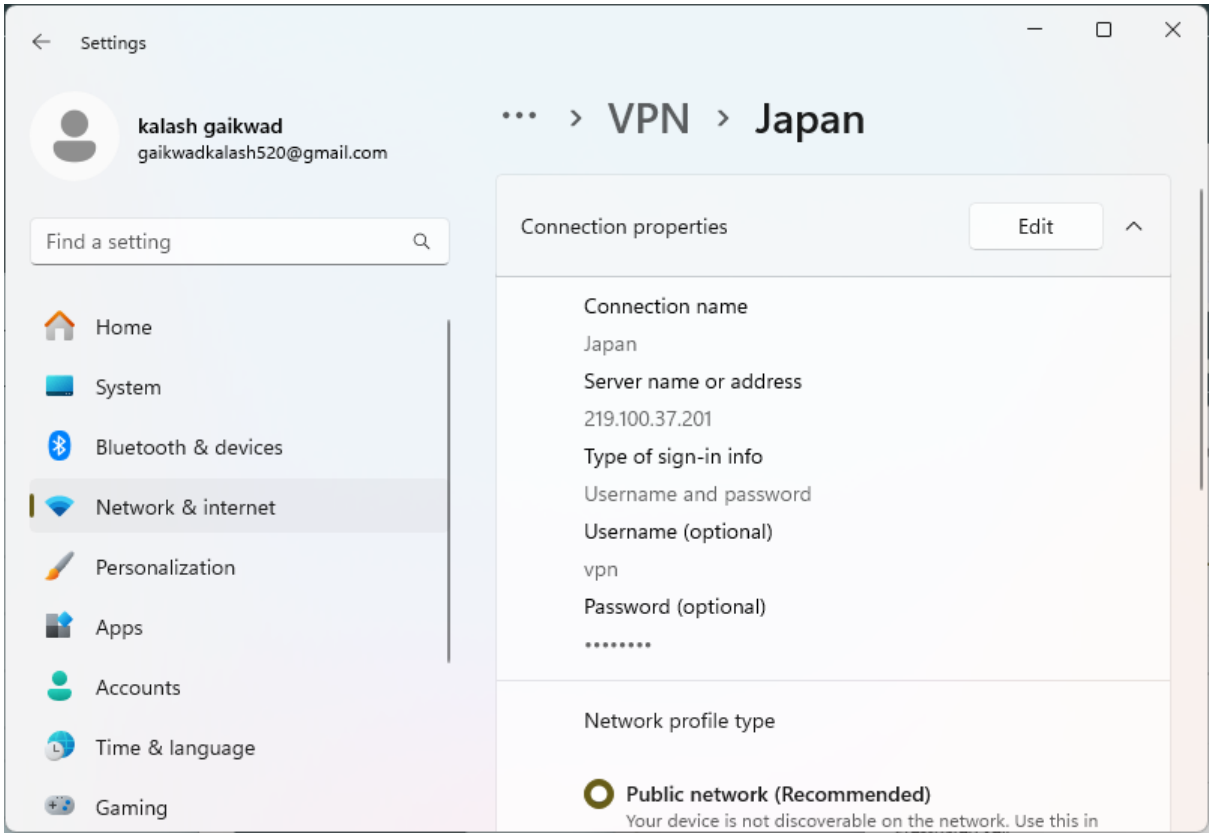
INS PRACTICAL 2025-26

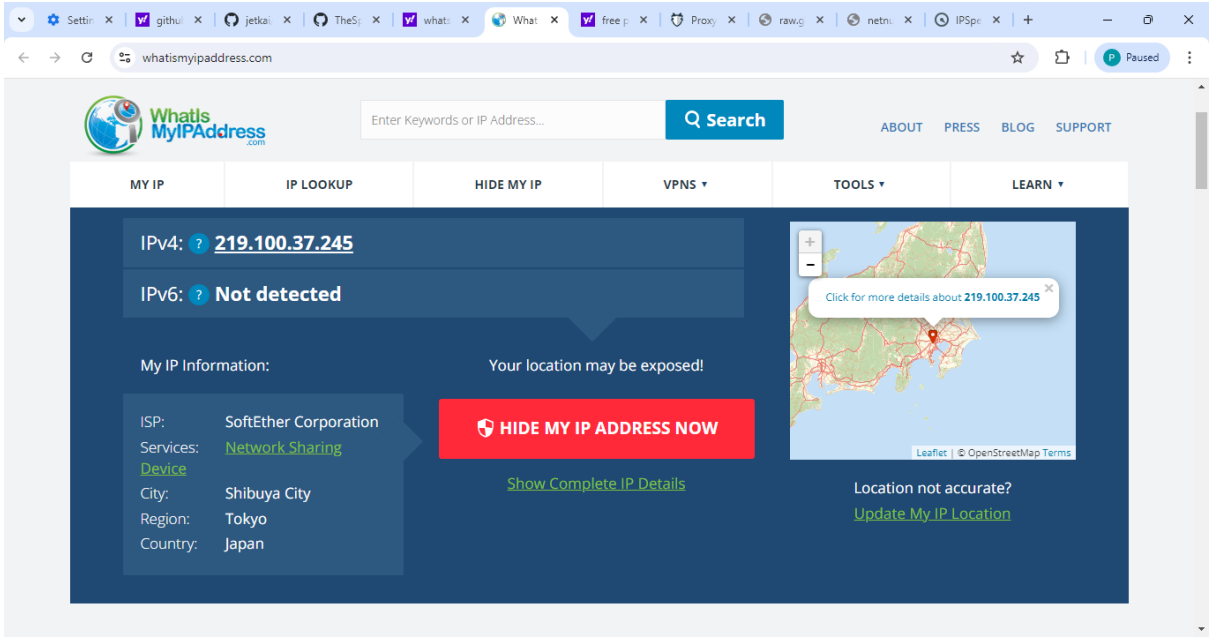




Name: Rudra Dubey Roll no: 06 Class: TYCS

INS PRACTICAL 2025-26





Name: Rudra Dubey Roll no: 06 Class: TYCS

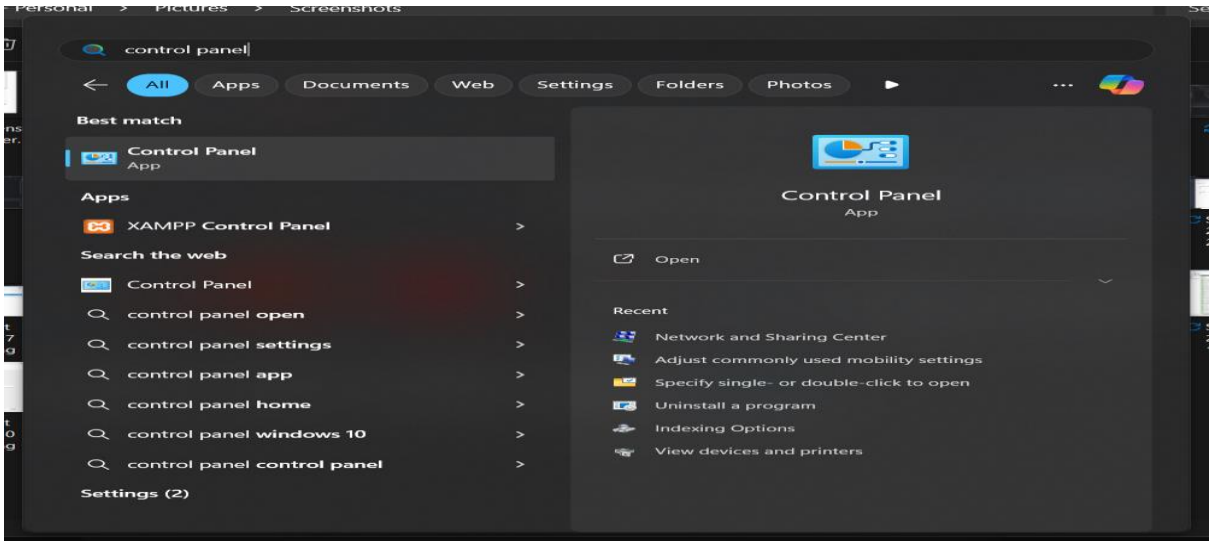
INS PRACTICAL 2025-26

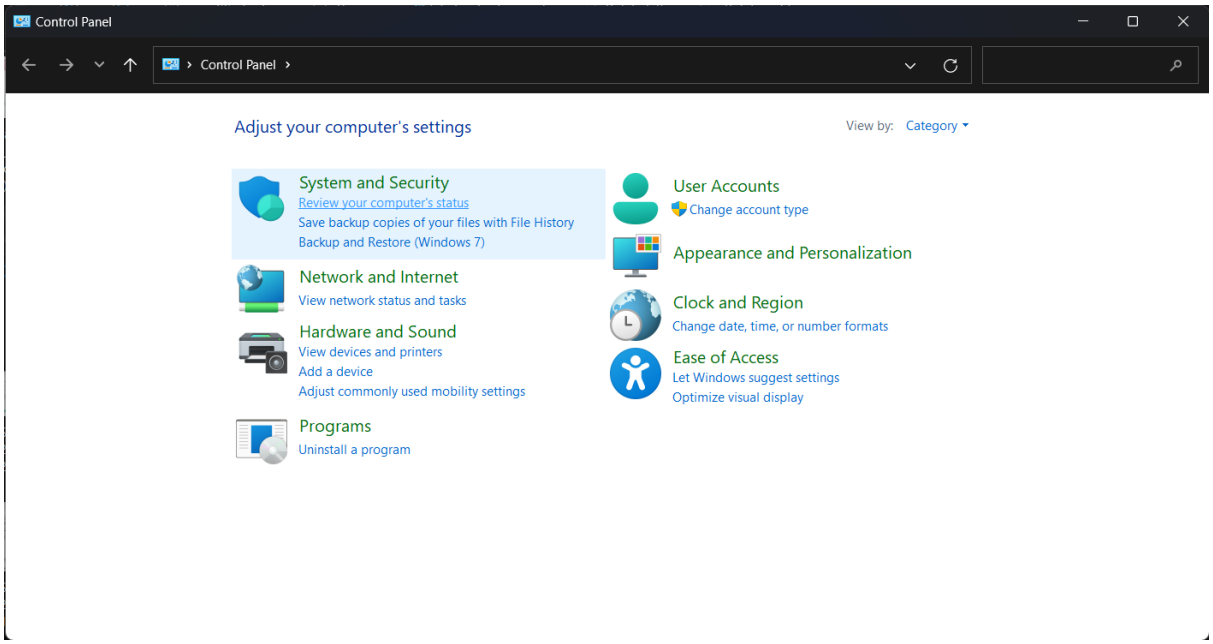
Practical 8

Q. Firewall Configuration and Rule-based Filtering

Configure and test firewall rules to control network traffic, filter packets based on specified criteria, and protect network resources from unauthorized access.

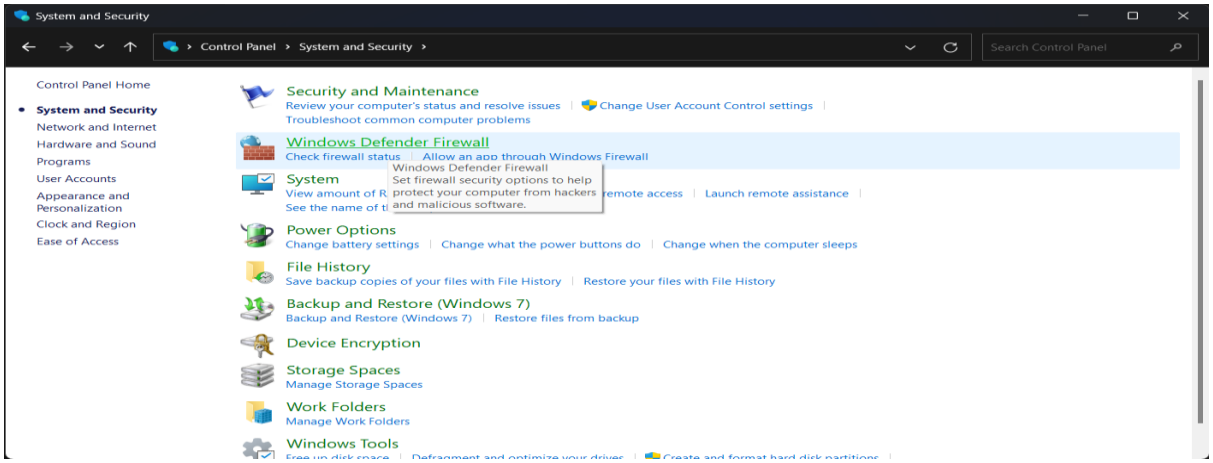
Start → Control Panel → Windows Firewall → Advanced Settings

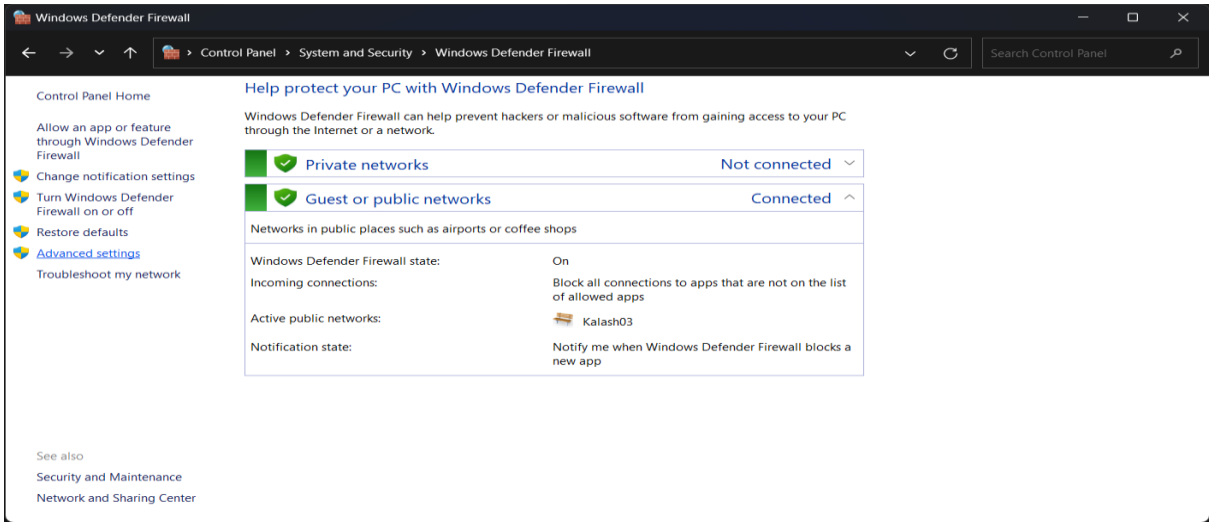


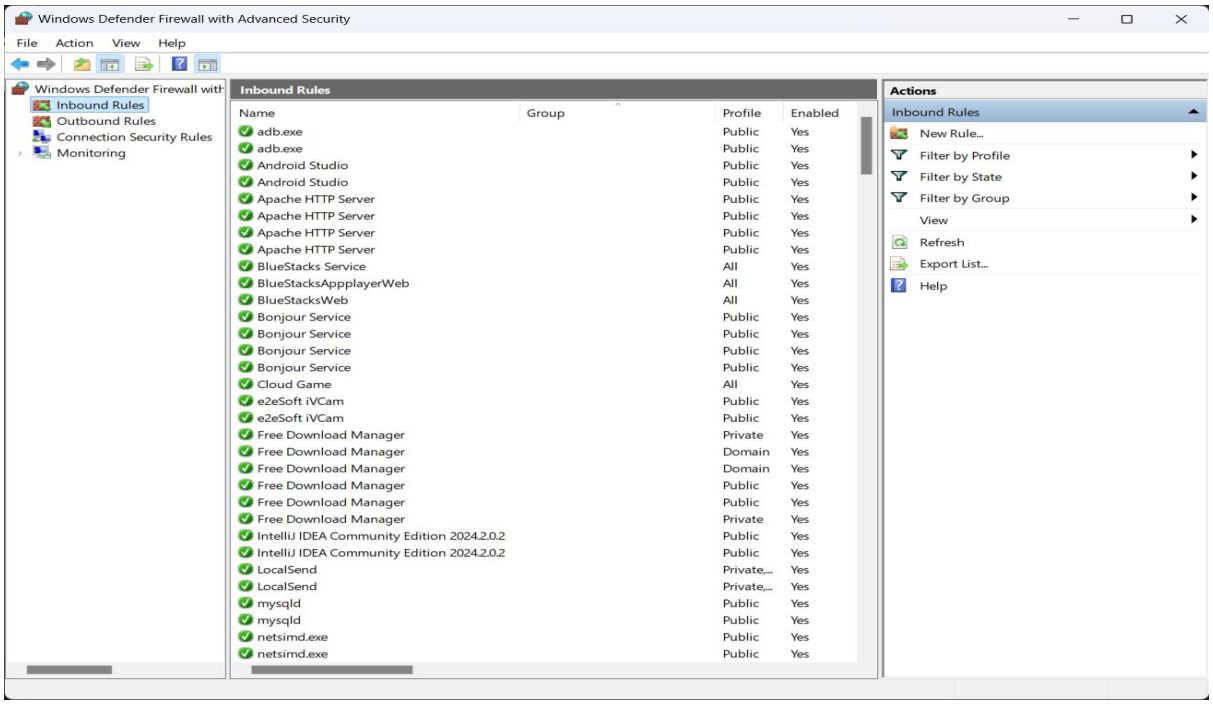


Name: Rudra Dubey Roll no: 06 Class: TYCS

INS PRACTICAL 2025-26

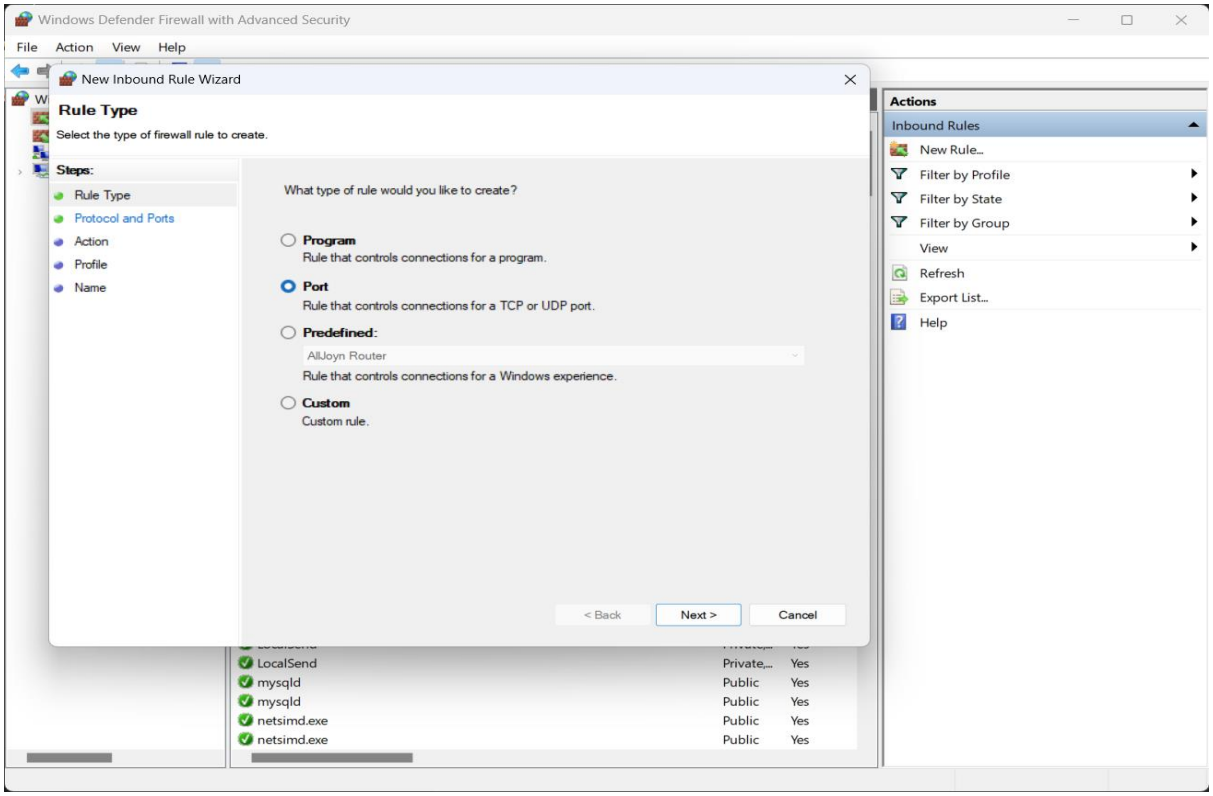


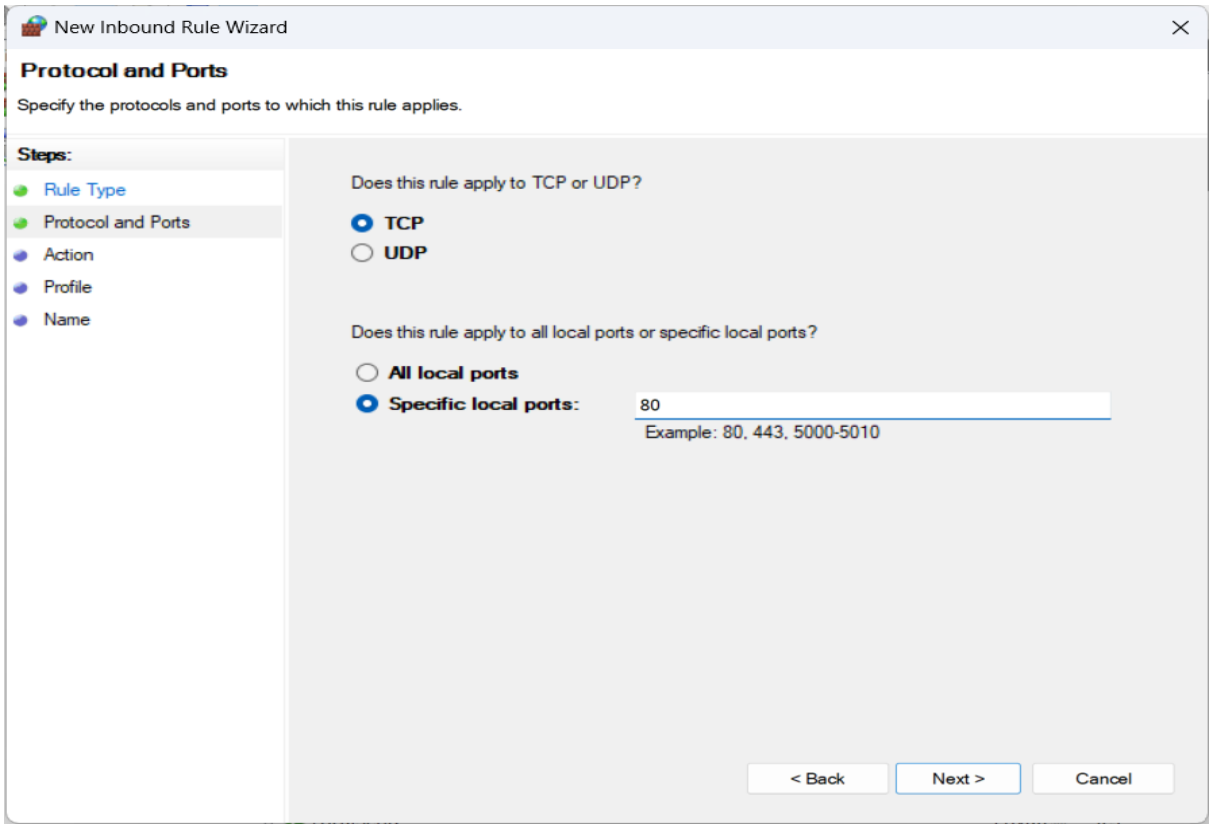




Name: Rudra Dubey Roll no: 06 Class: TYCS

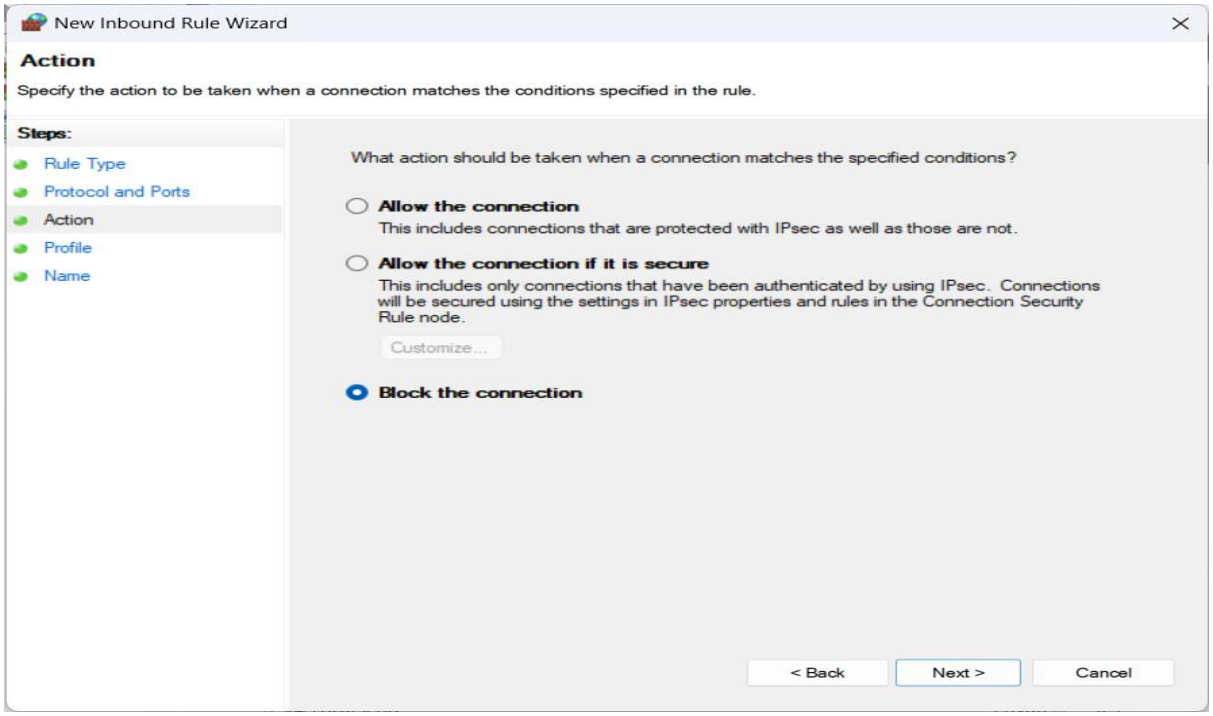
INS PRACTICAL 2025-26

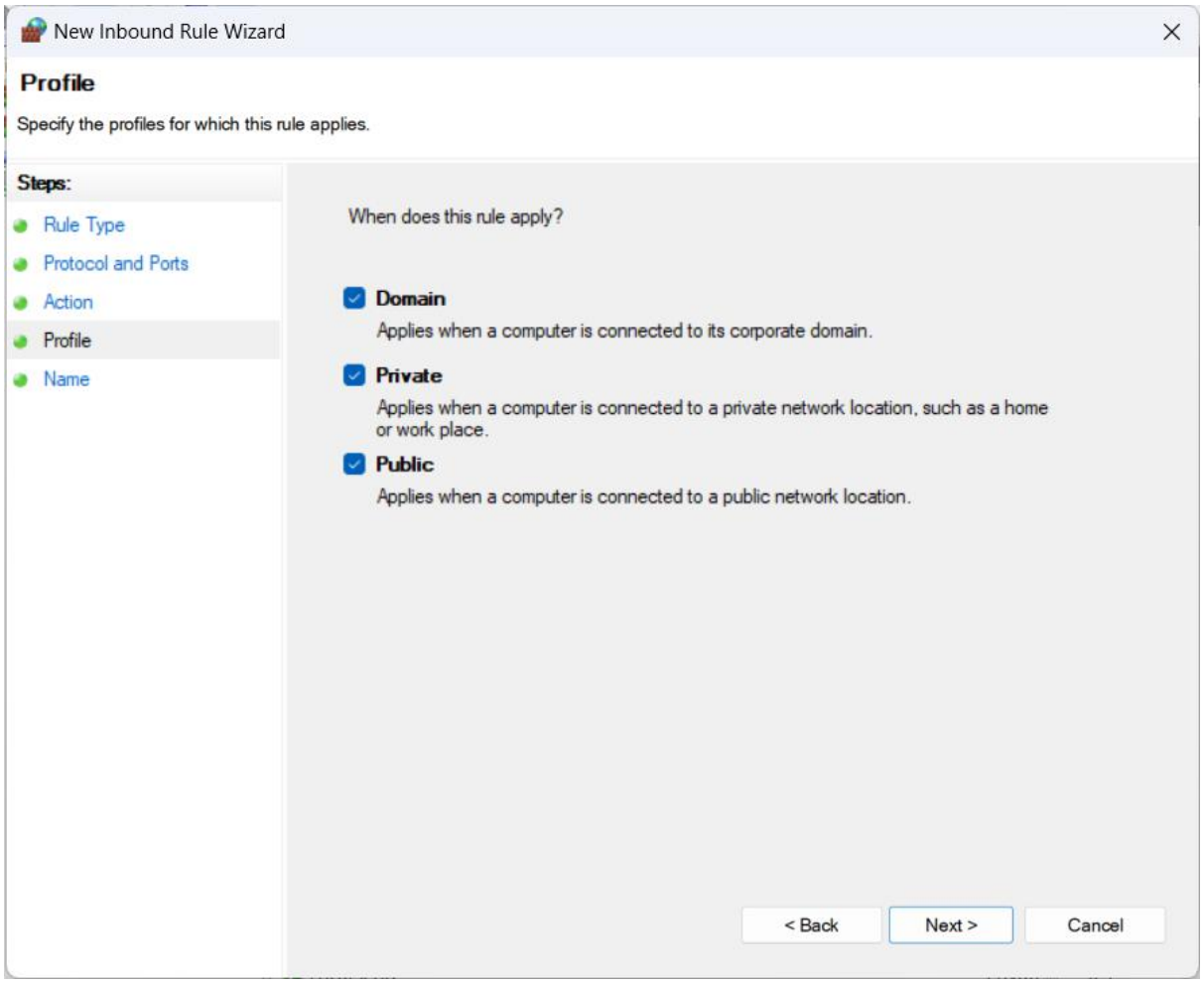




Name: Rudra Dubey Roll no: 06 Class: TYCS

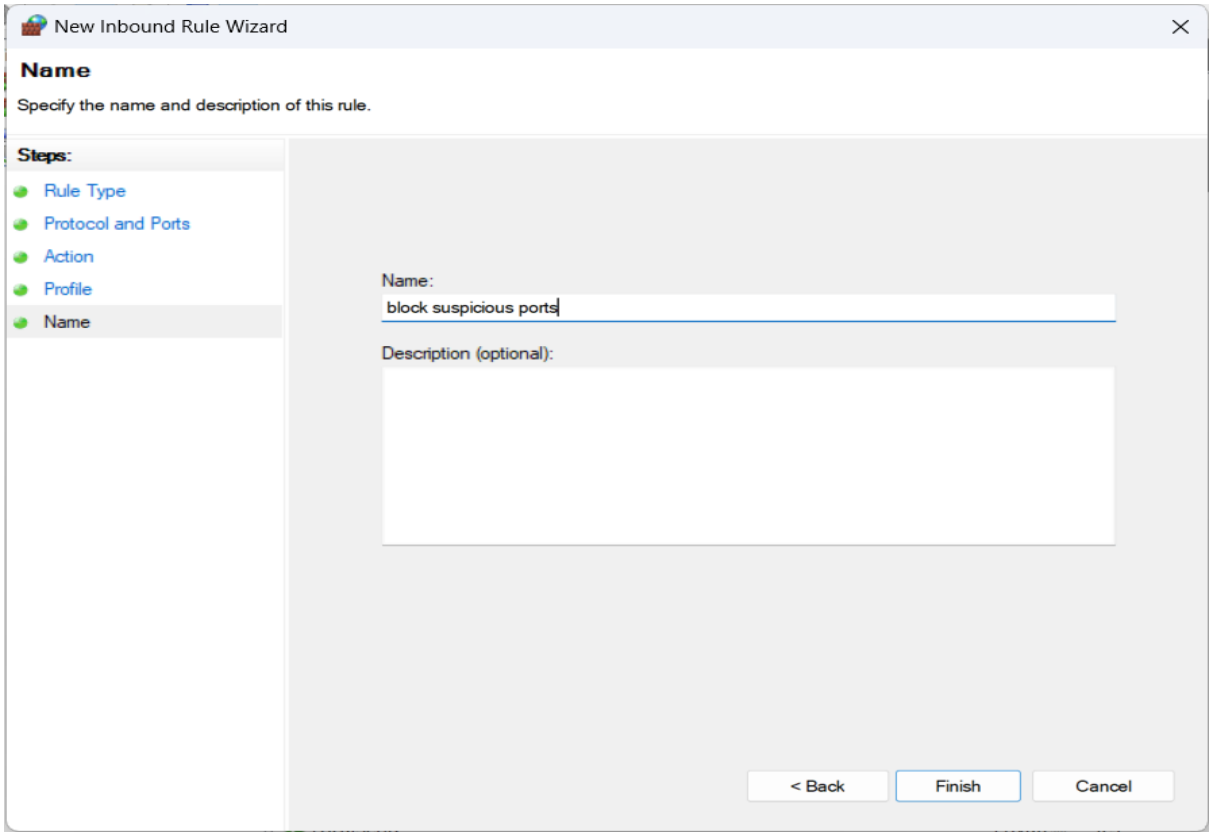
INS PRACTICAL 2025-26

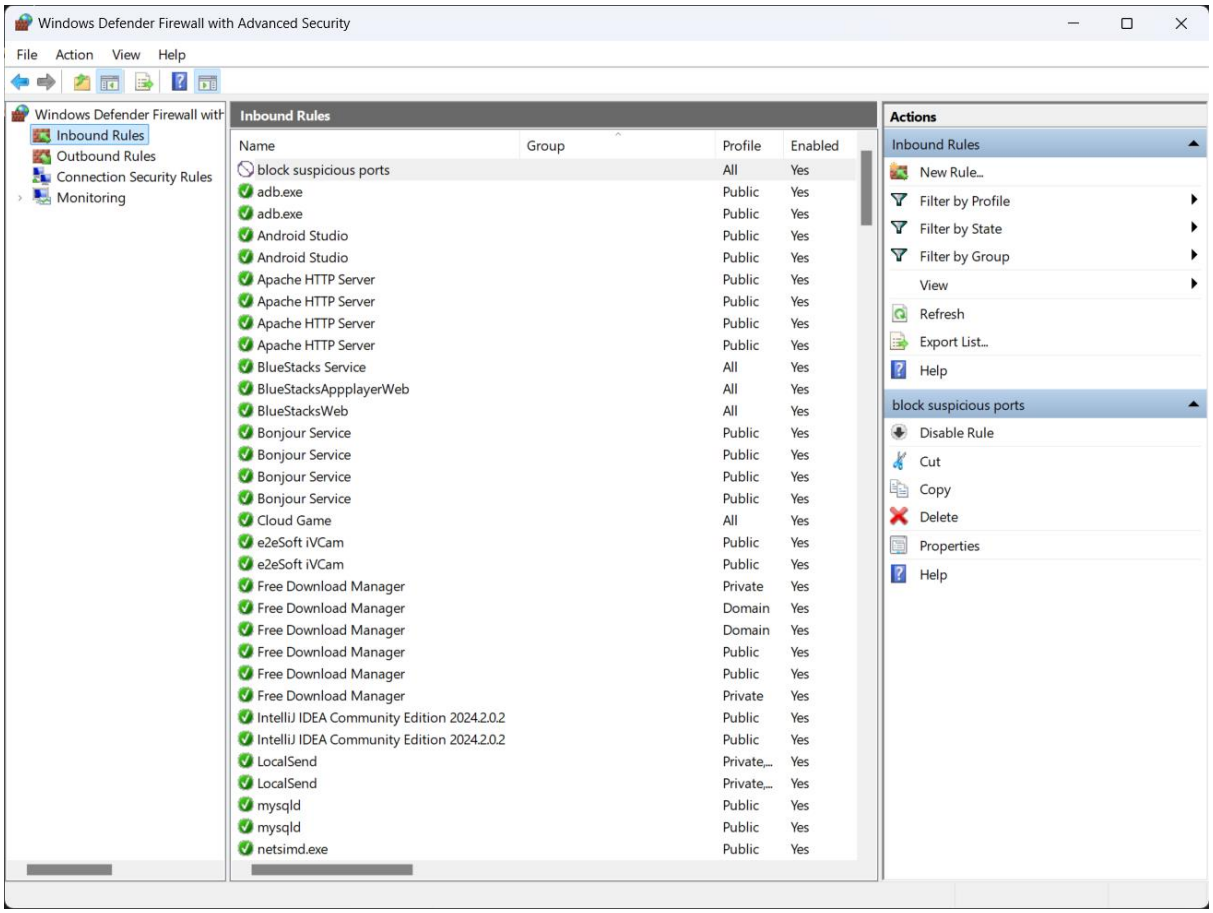




Name: Rudra Dubey Roll no: 06 Class: TYCS

INS PRACTICAL 2025-26





Name: Rudra Dubey Roll no: 06 Class: TYCS